INSTITUTO TECNOLOGICO SUPERIOR
ITESA
DEL ORIENTE DEL ESTADO DE HIDALGO

Ingeniería en Sistemas Computacionales

MÉTODOS NUMÉRICOS

Diferenciación e integración númerica

Métodos:

Regla del trapecio

Regla de Simpson 1/3

Regla de Simpson 3/8

Cuadratura Gaussiana

Regla de los 3 puntos

Regla de los 5 puntos

Contenido

..... 1
Introducción 4
Desarrollo de Ejercicios por Método 5
Regla del Trapecio 5
Algoritmo 5
Caso Ideal (Función Lineal) 5
Caso con Error 5
Caso Decidido - Aplicación 5
Códigos 5
Caso Ideal 5
Caso con Error 6
Caso Decidido 6
Regla de Simpson (1/3) 7
Algoritmo 7
Caso Ideal 7
Caso con Error 7
Caso Decidido 7
Código 7
Caso Ideal 7
Caso con Error 8
Caso Decidido 8
Regla de Simpson (3/8) 9
Algoritmo 9
Caso Ideal 9
Caso con Error 9
Caso Decidido 9
Código 9
Caso Ideal 9
Caso con Error 10
Caso Decidido 10
Cuadratura Gaussiana 11
Algoritmo 11
Caso Ideal 11
Caso con Error 11
Caso Decidido 11
Código 11
Caso Ideal 11
Caso con Error 12

Caso Decidido 12
Regla de los 3 puntos 13
Algoritmo 13
Caso Ideal 13
Caso con Error 13
Caso Decidido 13
Código 13
Caso Ideal 13
Caso con Error 14
Caso Decidido 14
Regla de los 5 puntos 15
Algoritmo 15
Caso Ideal 15
Caso con Error 15
Caso Decidido 15
Código 15
Caso Ideal 15
Caso con Error 16
Caso Decidido 16
Conclusión 16

Introducción

La integración numérica consiste en encontrar una aproximación al valor de una integral definida cuando no es posible hallar la antiderivada de forma analítica o cuando la función está definida por un conjunto de puntos discretos. En este reporte se analizan los métodos fundamentales, evaluando su precisión y limitaciones.

Desarrollo de Ejercicios por Método

Regla del Trapecio

Algoritmo

Este método aproxima el área bajo la función f(x) conectando los puntos con una línea recta, formando un trapecio.

1. Define el intervalo $[a, b]$.
2. Calcula el ancho del intervalo: $h = b - a$.
3. Aplica la fórmula:

$$I \approx \frac{h}{2}[f(a) + f(b)]$$

4. Para mayor precisión (Trapecio Compuesto), divide el intervalo en n segmentos y suma las áreas.

Caso Ideal (Función Lineal)

- **Ejercicio 1 (Caso Ideal):** $f(x) = 5x + 2$ en $[0, 2]$. (Funciones lineales).

Caso con Error

- **Ejercicio 2 (Caso con Error):** $f(x) = \sin(x^2)$ en $[0, 2]$. (Alta oscilación en intervalo amplio).

Caso Decidido - Aplicación

- **Ejercicio 3 (Caso Decidido - Aplicación):** $f(x) = \sqrt{1 + x^2}$ en $[0, 1]$. (Longitud de arco).

Códigos

Caso Ideal

```java
public class TrapecioIdeal {
    public static void main(String[] args) {
        // f(x) = 5x + 2 en [0, 2]
        double a = 0, b = 2;
        double resultado = ((b - a) / 2) * ((5 * a + 2) + (5 * b + 2));
        System.out.println("Resultado Trapecio Ideal: " + resultado);
    }
}
// Interpretación: El error es 0 porque el método aproxima con una línea recta y la función YA es una línea recta.
```

```
Resultado Trapecio Ideal: 14.0

=== Code Execution Successful ===
```

Caso con Error

```java
public class TrapecioError {
    public static void main(String[] args) {
        // f(x) = sin(x^2) en [0, 2]
        double a = 0, b = 2;
        double fa = Math.sin(a * a);
        double fb = Math.sin(b * b);
        double resultado = ((b - a) / 2) * (fa + fb);
        System.out.println("Resultado Trapecio Error: " + resultado);
    }
}
// Interpretación: El error es alto porque entre 0 y 2 la función sube y baja varias veces, y un solo trapecio ignora esas curvas.
```

```
ERROR!
Resultado Trapecio Error: -0.7568024953079282

=== Code Execution Successful ===
```

Caso Decidido

```java
public class TrapecioAplicado {
    public static void main(String[] args) {
        // f(x) = sqrt(1 + x^2) en [0, 1]
        double a = 0, b = 1;
        double fa = Math.sqrt(1 + a * a);
        double fb = Math.sqrt(1 + b * b);
        double resultado = ((b - a) / 2) * (fa + fb);
        System.out.println("Resultado Trapecio Aplicado: " + resultado);
    }
}
// Interpretación: Útil para aproximaciones rápidas en ingeniería donde no se requiere precisión extrema.
```

```
Resultado Trapecio Aplicado: 1.2071067811865475

=== Code Execution Successful ===
```

Regla de Simpson (1/3)

Algoritmo

Utiliza un polinomio de segundo grado (una parábola) para conectar tres puntos. Es mucho más preciso que el trapecio.

1. Requiere **3 puntos** (un número par de intervalos, $n = 2$).
2. Calcula el paso: $h = \frac{b-a}{2}$.
3. Identifica los puntos: x_0, x_1, x_2.
4. Aplica la fórmula:

$$I \approx \frac{h}{3}[f(x_0) + 4f(x_1) + f(x_2)]$$

Caso Ideal

Algoritmo: $\frac{h}{3}[f(a) + 4f(m) + f(b)]$ donde m es el punto medio.

- **Ejercicio 4 (Caso Ideal):** $f(x) = x^3 - 2x^2$ en $[0, 2]$. (Exacto para polinomios grado ≤ 3).

Caso con Error

- **Ejercicio 5 (Caso con Error):** $f(x) = 1/x$ en $[0, 1]$. (Singularidad en el límite inferior).

Caso Decidido

- **Ejercicio 6 (Caso Decidido - Probabilidad):** $f(x) = e^{-x^2}$ en $[0, 1]$. (Distribución normal).

Código

Caso Ideal

```java
public class Simpson13Ideal {
    public static void main(String[] args) {
        // f(x) = x^3 en [0, 2]
        double a = 0, b = 2, m = (a + b) / 2;
        double h = (b - a) / 2;
        double res = (h / 3) * (Math.pow(a, 3) + 4 * Math.pow(m, 3) + Math.pow(b, 3));
        System.out.println("Resultado Simpson 1/3 Ideal: " + res);
    }
}
// Interpretación: Da el valor exacto (4.0) porque Simpson es preciso hasta polinomios de tercer grado.
```

```
Resultado Simpson 1/3 Ideal: 4.0

=== Code Execution Successful ===
```

Caso con Error

```java
public class Simpson13Error {
    public static void main(String[] args) {
        // f(x) = 1/x en [0, 1]
        double a = 0, b = 1, m = 0.5;
        double res = (1.0/6.0) * ((1/a) + 4*(1/m) + (1/b));
        System.out.println("Resultado Simpson 1/3 Error: " + res);
    }
}
// Interpretación: El resultado es "Infinity". El método falla si la función no es continua en el intervalo.
```

```
ERROR!
Resultado Simpson 1/3 Error: Infinity

=== Code Execution Successful ===
```

Caso Decidido

```java
public class Simpson13Aplicado {
    public static void main(String[] args) {
        // f(x) = exp(-x^2) en [0, 1]
        double a = 0, b = 1, m = 0.5;
        double res = (1.0/6.0) * (Math.exp(-a*a) + 4*Math.exp(-m*m) + Math.exp(-b*b));
        System.out.println("Resultado Simpson 1/3 Aplicado: " + res);
    }
}
// Interpretación: Muy preciso para calcular áreas bajo la campana de Gauss en estadística.
```

```
Resultado Simpson 1/3 Aplicado: 0.7471804289095104

=== Code Execution Successful ===
```

Regla de Simpson (3/8)

Algoritmo

Similar a la anterior, pero utiliza un polinomio de tercer grado para conectar cuatro puntos.

1. Requiere **4 puntos** (intervalos múltiples de 3).
2. Calcula el paso: $h = \frac{b-a}{3}$.
3. Aplica la fórmula:

$$I \approx \frac{3h}{8}[f(x_0) + 3f(x_1) + 3f(x_2) + f(x_3)]$$

Nota: Se usa cuando el número de subdivisiones es impar y no puedes usar Simpson 1/3 puramente.

Caso Ideal

Algoritmo: $\frac{3h}{8}[f(x_0) + 3f(x_1) + 3f(x_2) + f(x_3)]$

- **Ejercicio 7 (Caso Ideal):** $f(x) = x^3 + 5$ en $[1, 4]$.

Caso con Error

- **Ejercicio 8 (Caso con Error):** $f(x) = |x - 2|$ en $[0, 4]$. (No derivable en el pico, Simpson asume suavidad).

Caso Decidido

- **Ejercicio 9 (Caso Decidido - Logarítmico):** $f(x) = \ln(x)$ en $[1, 2]$.

Código

Caso Ideal

```java
public class Simpson38Ideal {
    public static void main(String[] args) {
        double a = 1, b = 4, h = (b - a) / 3;
        // f(x) = x^3 + 5
        double res = (3 * h / 8) * (f(a) + 3*f(a+h) + 3*f(a+2*h) + f(b));
        System.out.println("Resultado Simpson 3/8 Ideal: " + res);
    }
    public static double f(double x) { return Math.pow(x, 3) + 5; }
}
```

```
Resultado Simpson 3/8 Ideal: 78.75

=== Code Execution Successful ===
```

Caso con Error

```java
public class Simpson38Error {
    public static void main(String[] args) {
        double a = 0, b = 4, h = (b-a)/3;
        // f(x) = |x - 2| (Pico en x=2)
        double res = (3 * h / 8) * (f(a) + 3*f(a+h) + 3*f(a+2*h) + f(b));
        System.out.println("Resultado Simpson 3/8 Error: " + res);
    }
    public static double f(double x) { return Math.abs(x - 2); }
}
// Interpretación: El error ocurre porque Simpson asume que la función es una curva suave, no un ángulo agudo.
```

```
ERROR!
Resultado Simpson 3/8 Error: 4.0

=== Code Execution Successful ===
```

Caso Decidido

```java
public class Simpson38Aplicado {
    public static void main(String[] args) {
        double a = 1, b = 2, h = (b-a)/3;
        // f(x) = ln(x)
        double res = (3 * h / 8) * (Math.log(a) + 3*Math.log(a+h) + 3*Math.log(a+2*h) + Math.log(b));
        System.out.println("Resultado Simpson 3/8 Aplicado: " + res);
    }
```

```
Resultado Simpson 3/8 Aplicado: 0.3860837836516575

=== Code Execution Successful ===
```

Cuadratura Gaussiana

Algoritmo

A diferencia de los anteriores, no usa puntos fijos (como los extremos). Selecciona puntos óptimos para maximizar la precisión.

1. Cambia la variable de $x \in [a, b]$ a $t \in [-1, 1]$ usando: $x = \frac{b-a}{2}t + \frac{b+a}{2}$.
2. Para 2 puntos, los pesos son $w_1 = w_2 = 1$ y los puntos son $t = \pm\frac{1}{\sqrt{3}}$.
3. Aplica:

$$I \approx \frac{b-a}{2}[f(x(t_1)) + f(x(t_2))]$$

Caso Ideal

Algoritmo: Transformar $[a, b] \to [-1, 1]$ y evaluar $w_1 f(z_1) + w_2 f(z_2)$.

- **Ejercicio 10 (Caso Ideal):** $f(x) = x^2$ en $[-1, 1]$.

Caso con Error

- **Ejercicio 11 (Caso con Error):** $f(x) = \sqrt[3]{x}$ en $[-1, 1]$. (Pendiente infinita en el origen).

Caso Decidido

- **Ejercicio 12 (Caso Decidido - Trigonométrico):** $f(x) = \cos(x)$ en $[0, \pi/2]$.

Código

Caso Ideal

```java
public class GaussIdeal {
    public static void main(String[] args) {
        // f(x) = x^2 en [-1, 1]
        double x1 = -0.57735, x2 = 0.57735;
        double res = (1.0 * (x1*x1)) + (1.0 * (x2*x2));
        System.out.println("Gauss Ideal: " + res);
    }
}
```

```
Gauss Ideal: 0.666666045

=== Code Execution Successful ===
```

Caso con Error

```java
public class GaussError {
    public static void main(String[] args) {
        // f(x) = x^(1/3) en [-1, 1]. Pendiente infinita en 0.
        double x1 = -0.57735, x2 = 0.57735;
        double res = Math.pow(x1, 1.0/3.0) + Math.pow(x2, 1.0/3.0);
        System.out.println("Gauss Error: " + res);
    }
}
```

```
ERROR!
Gauss Error: NaN

=== Code Execution Successful ===
```

El método da "error" (en términos de fiabilidad) porque la función no es analítica (suave) en todo el intervalo. La Cuadratura Gaussiana odia las funciones con "picos" o pendientes infinitas.

Caso Decidido

```java
public class GaussAplicado {
    public static void main(String[] args) {
        double a = 0, b = Math.PI/2;
        double x1 = -0.57735, x2 = 0.57735;
        double c1 = (b-a)/2, c2 = (b+a)/2;
        double res = c1 * (Math.cos(c1*x1+c2) + Math.cos(c1*x2+c2));
        System.out.println("Gauss Aplicado: " + res);
    }
}
```

```
Gauss Aplicado: 0.9984727162757973

=== Code Execution Successful ===
```

Regla de los 3 puntos

Algoritmo

Se usa para calcular la **derivada** $f'(x)$ usando información de puntos vecinos.

Algoritmo (Diferencia central):

1. Toma un punto anterior $x - h$, el punto central x, y uno siguiente $x + h$.
2. Aplica la fórmula para reducir el error:

$$f'(x) \approx \frac{f(x+h) - f(x-h)}{2h}$$

Caso Ideal

Algoritmo: $f'(x) \approx \frac{f(x+h)-f(x-h)}{2h}$

- **Ejercicio 13 (Caso Ideal):** $f(x) = x^2$ en $x = 2, h = 0.1$.

Caso con Error

- **Ejercicio 14 (Caso con Error):** $f(x) = \tan(x)$ cerca de $\pi/2$. (Crecimiento asintótico).

Caso Decidido

- **Ejercicio 15 (Caso Decidido):** $f(x) = \ln(x)$ en $x = 1$.

Código

Caso Ideal

```java
public class Derivada3Ideal {
    public static void main(String[] args) {
        // f(x) = x^2, x=2, h=0.1. Exacto: 4.0
        double x = 2, h = 0.1;
        double res = (Math.pow(x+h, 2) - Math.pow(x-h, 2)) / (2*h);
        System.out.println("Derivada 3P Ideal: " + res);
    }
}
```

```
Derivada 3P Ideal: 4.000000000000001

=== Code Execution Successful ===
```

Caso con Error

```java
public class Derivada3Error {
    public static void main(String[] args) {
        // f(x) = tan(x) cerca de pi/2
        double x = 1.57, h = 0.01;
        double res = (Math.tan(x+h) - Math.tan(x-h)) / (2*h);
        System.out.println("Derivada 3P Error: " + res);
    }
}
```

```
ERROR!
Derivada 3P Error: -10063.484996077383

=== Code Execution Successful ===
```

Caso Decidido

```java
public class Derivada3Aplicado {
    public static void main(String[] args) {
        double x = 1.0, h = 0.1;
        double res = (Math.log(x+h) - Math.log(x-h)) / (2*h);
        System.out.println("Derivada 3P Aplicado: " + res);
    }
}
```

```
Derivada 3P Aplicado: 1.003353477310756

=== Code Execution Successful ===
```

Regla de los 5 puntos

Algoritmo

Es una extensión de la anterior que ofrece una precisión de orden superior (h^4) al usar más información de la vecindad.

1. Evalúa la función en 5 puntos: $x - 2h, x - h, x, x + h, x + 2h$.
2. Aplica la fórmula ponderada:

$$f'(x) \approx \frac{-f(x+2h) + 8f(x+h) - 8f(x-h) + f(x-2h)}{12h}$$

Caso Ideal

Algoritmo: $f'(x) \approx \frac{-f(x+2h)+8f(x+h)-8f(x-h)+f(x-2h)}{12h}$

- **Ejercicio 16 (Caso Ideal):** $f(x) = x^4$ en $x = 1$. (Exacto para grado alto).

Caso con Error

- **Ejercicio 17 (Caso con Error):** $f(x) = e^x$ con $h = 1.0$ (paso demasiado grande).

Caso Decidido

- **Ejercicio 18 (Caso Decidido):** $f(x) = \sin(x)$ en $x = 0$.

Código

Caso Ideal

```java
public class Derivada5Ideal {
    public static void main(String[] args) {
        // f(x) = x^4, x=1, h=0.1
        double x = 1, h = 0.1;
        double res = (-f(x+2*h) + 8*f(x+h) - 8*f(x-h) + f(x-2*h)) / (12*h);
        System.out.println("Derivada 5P Ideal: " + res);
    }
    public static double f(double x) { return Math.pow(x, 4); }
}
```

```
Derivada 5P Ideal: 4.000000000000002

=== Code Execution Successful ===
```

Caso con Error

```java
public class Derivada5Error {
    public static void main(String[] args) {
        // f(x) = exp(x), h=2.0 (Demasiado grande)
        double x = 1, h = 2.0;
        double res = (-Math.exp(x+2*h) + 8*Math.exp(x+h) - 8*Math.exp(x-h) + Math.exp(x-2*h))
            / (12*h);
        System.out.println("Derivada 5P Error: " + res);
    }
}
```

```
ERROR!
Derivada 5P Error: 0.3907453259133779

=== Code Execution Successful ===
```

Caso Decidido

```java
public class Derivada5Aplicado {
    public static void main(String[] args) {
        double x = 0, h = 0.1;
        double res = (-Math.sin(x+2*h) + 8*Math.sin(x+h) - 8*Math.sin(x-h) + Math.sin(x-2*h))
            / (12*h);
        System.out.println("Derivada 5P Aplicado: " + res);
    }
}
```

```
Derivada 5P Aplicado: 0.9999966706326063

=== Code Execution Successful ===
```

Conclusión

La integración y diferenciación numérica permiten resolver problemas donde no existe una fórmula analítica, destacando la Regla del Trapecio por su sencillez y la Regla de Simpson (1/3 y 3/8) por su gran equilibrio entre precisión y complejidad técnica, para máxima eficiencia computacional en funciones suaves, la Cuadratura Gaussiana es la opción superior al optimizar la posición de los puntos de evaluación, mientras que las reglas de 3 y 5 puntos son fundamentales para estimar pendientes con alta fidelidad, recomiendo usar Simpson para datos experimentales tabulados y Gauss para simulaciones de software, siempre vigilando que la función no presente picos o discontinuidades que invaliden los resultados, la elección del método depende de si priorizamos la facilidad de implementación, la rapidez de procesamiento o la exactitud en el manejo de curvas complejas.